



Heuristic approach for the FSTSP

by

Rafael Hamelink (SNR: 2061119)

A thesis submitted in partial fulfillment of the requirements for the degree of
Bachelor in Econometrics and Operations Research

Tilburg School of Economics and Management
Tilburg University

Supervised by: dr.ir.ing. René Peeters

Date: 25/4/2026

Contents

1	Introduction	2
2	Related literature	4
2.1	Extensions and alternative models	4
2.2	Operational considerations	5
2.3	Exact solution approaches	5
2.4	Heuristic approaches	5
2.5	Positioning of this study	6
3	Preliminaries	7
3.1	The Classical Traveling Salesman Problem	7
3.2	Extension to the Flying Sidekick Traveling Salesman Problem .	8
3.2.1	Model notation	8
4	Heuristics	11
4.1	Route and Reassign Heuristic	11
4.1.1	Complete algorithmic framework	12
4.1.2	Explanation of the heuristic	14
4.1.3	Example	15
4.2	Simulated Annealing heuristic	18
4.2.1	Solution representation	19
4.2.2	Main algorithm	20
4.2.3	Move operators	22
4.2.4	Feasibility check and evaluation	23
4.3	Hybrid Simulated Annealing heuristic	25
5	Experimental Setup	27
5.1	Base model	27
5.2	Simulated Annealing	28
6	Results	29
7	Conclusion	30
A	Extra Tables and Figures	33

Chapter 1

Introduction

In recent years, the rapid development of drone technology has created new opportunities within the logistics sector. Since 2013, major companies such as UPS, Amazon, and DHL have actively experimented with the use of Unmanned Aerial Vehicles (UAVs) for parcel delivery (Stern, J., 2013). Drones offer several potential advantages, including faster point-to-point delivery, reduced dependence on road infrastructure, and increased flexibility in reaching remote or congested areas. As a result, they are particularly promising for improving efficiency in last-mile delivery, which is often the most time-consuming and costly segment of the logistics chain.

The integration of UAVs into parcel delivery systems naturally raises the question of how such operations can be planned and optimized. Some routing problems have long been modeled using the well-known Traveling Salesman Problem (TSP). In its classical form, the TSP considers a single vehicle that must visit a set of customers exactly once and return to the depot, while minimizing the total travel cost or time. This problem has been extensively studied and serves as a foundational model for a wide range of vehicle routing applications.

However, the introduction of drones fundamentally alters the structure of the problem. Instead of a single delivery vehicle, the system now consists of a truck and one or more UAVs that can operate in coordination. Consequently, the classical TSP is no longer sufficient to accurately capture the operational complexity of such systems.

To address this, Murray and Chu (Murray and Chu, 2015) introduced the Flying Sidekick Traveling Salesman Problem (FSTSP), a generalization of the TSP in which a delivery truck and a drone work collaboratively to serve customers. In this setting, the truck follows a route similar to the traditional TSP, while the drone can be dispatched from the truck to serve individual customers and rendezvous with the truck at a next location. The objective is to minimize the total delivery completion time, taking into account both vehicle routing and the synchronization constraints between the truck and the drone.

The FSTSP thus extends the classical TSP by incorporating vehicle coordination, timing dependencies, and additional feasibility constraints such as drone endurance. These added complexities exponentially increase the diffi-

culty of the problem, making exact solution methods computationally challenging for larger instances and motivating the development of heuristic approaches.

Therefore, this study aims to investigate a (or maybe two if I feel even more ambitious) heuristic approach to minimize the total completion time of a full delivery tour within reasonable computational time.

Chapter 2

Related literature

The integration of drones into logistics has led to a rapidly expanding body of research on coordinated truck–drone delivery systems. This section reviews the relevant literature, focusing on different modeling approaches, problem variants, and solution methods.

2.1 Extensions and alternative models

Murray and Chu (Murray and Chu, 2015) introduced one of the first models combining truck and drone delivery within a TSP-based framework. In addition to the Flying Sidekick Traveling Salesman Problem (FSTSP), they also proposed the Parallel Drone Scheduling Traveling Salesman Problem (PDSTSP). In this variant, drones operate directly from the depot and serve customers independently within their flight range, while the truck delivers the remaining parcels. This removes the need for synchronization between truck and drone during routing, but introduces constraints related to drone range and depot-based operations.

Building on this line of research, Agatz et al. (Agatz et al., 2018) studied related problems such as the Traveling Salesman Problem with Drone (TSPD) and the Traveling Salesman Problem with Multiple Customers (TSP-MC). These models differ from the FSTSP in the way the drone interacts with the truck, for instance by allowing alternative coordination mechanisms or permitting the drone to serve multiple customers within a single trip. While closely related, these formulations involve different structural assumptions and therefore require distinct solution approaches.

A further extension involves the use of multiple drones. In such settings, the truck can deploy more than one UAV, either sequentially or simultaneously. While this increases operational flexibility, it also introduces additional complexity in terms of scheduling and synchronization. Studies such as Salama et al. (Salama and Srinivas, 2022) and Najy et al. (Najy, 2023) investigate these multi-drone systems, often relying on heuristic or metaheuristic methods due to the increased computational difficulty.

2.2 Operational considerations

To make truck–drone models applicable in practice, a variety of operational factors must be taken into account. These include drone flight capabilities, energy consumption, and the influence of infrastructure on routing decisions. In particular, the effective travel time of a drone is not solely determined by distance, but also by acceleration, cruising speed, and deceleration phases.

More recent studies incorporate such operational aspects directly into routing models. Li et al. (Li, 2022) and De Kuyffer et al. (De Kuyffer, 2024) consider additional coordination constraints and more realistic system behavior, while Zhang et al. (Zhang, 2022) investigate multi-objective formulations that balance delivery time with other performance metrics such as energy consumption. These developments reflect a growing effort to bridge the gap between theoretical models and real-world applications.

2.3 Exact solution approaches

Since the introduction of the FSTSP, several studies have focused on improving exact solution methods. Early mixed-integer linear programming (MILP) formulations originally introduced by Murray and Chu (Murray and Chu, 2015) relied on time-indexed variables and Big-M constraints to model synchronization between the truck and the drone. While straightforward, these formulations often resulted in weak linear relaxations and poor computational performance.

To address these limitations, Boccia et al. (Boccia, 2023) proposed a stronger arc-based MILP formulation that models synchronization without relying on Big-M constraints. By using flow-based variables to capture the interaction between truck and drone, their formulation significantly improves computational performance and allows medium-sized instances to be solved more efficiently.

2.4 Heuristic approaches

Due to the inherent complexity of truck–drone routing problems, heuristic approaches play an important role. Murray and Chu (Murray and Chu, 2015) proposed a heuristic that starts from a truck-only TSP solution and iteratively improves it by reassigning customers. In each iteration, customers are either assigned to the drone or repositioned within the truck route based on potential time savings.

Es Yurek and Ozmutlu (Es Yurek and Ozmutlu, 2018) introduced a decomposition-based heuristic that separates routing and assignment decisions, allowing for improved scalability on larger instances. Boccia et al. (Boccia, 2023) also propose a heuristic approach in which computationally expensive components of the MILP, such as subtour elimination constraints, are relaxed and handled heuristically in order to reduce solution time while maintaining solution

quality.

Another heuristic approach uses Simulated Annealing (SA), introduced by (Kirkpatrick, 1983), which is an iterative metaheuristic that can be used to find good approximate solutions for difficult optimization problems. Within its core it considers changes that might increase the total completion time in its earlier steps in order to evaluate more possibilities. Multiple researchers such as (Ponza, 2016) have also implemented SA into the FSTSP.

2.5 Positioning of this study

This thesis focuses on the core routing and coordination problem of the FSTSP, while still incorporating limited operational elements. In particular, launch and recovery service times are included, and different relative speeds of the drone are evaluated. By doing so, the model maintains a balance between realism and computational tractability, allowing for a meaningful comparison between exact and heuristic solution approaches.

For the purposes of this study, the MILP formulation proposed by Boccia et al. (?) is implemented as the exact solution method. Although this formulation is more extensive than earlier models, it is specifically designed to improve computational performance by strengthening the underlying formulation.

In addition, the heuristic proposed by Murray and Chu (Murray and Chu, 2015) is selected. This choice is motivated by its conceptual simplicity, its direct applicability to the FSTSP, and thus suitability for implementation within the scope of this thesis. Moreover, it provides a clear and well-established baseline for comparing heuristic and exact solution approaches.

Chapter 3

Preliminaries

In this section both the TSP as well as the FSTSP will be defined and their mixed integer linear programming (MILP) will be formulated.

3.1 The Classical Traveling Salesman Problem

In the classical form of the TSP a salesman must visit each customer exactly once, while starting and ending at the depot, while minimizing the total travel cost. This must be done in one singular tour, thus subtours are not allowed.

Let $G = (V, A)$ be a directed graph, where $V = \{0, 1, \dots, n, n+1\}$ is the set of nodes, with customers $C = \{1, 2, \dots, n\}$ and $A = \{(i, j) : i, j \in V, i \neq j\}$ is the set of arcs. Node 0 and $n+1$ denote the starting and end point of the depot and nodes $1, \dots, n$ represent customers. For each arc $(i, j) \in A$, let c_{ij} denote the travel cost from node i to node j .

Define the binary decision variable:

$$x_{ij} = \begin{cases} 1 & \text{if the salesman travels from node } i \text{ to node } j, \\ 0 & \text{otherwise.} \end{cases}$$

The TSP can then be formulated as:

$$\min \sum_{i \in V} \sum_{\substack{j \in V \\ j \neq i}} c_{ij} x_{ij} \quad (3.1)$$

$$\text{s.t.} \quad \sum_{\substack{j \in V \\ j \neq i}} x_{ij} = 1 \quad \forall i \in V \quad (3.2)$$

$$\sum_{\substack{i \in V \\ i \neq j}} x_{ij} = 1 \quad \forall j \in V \quad (3.3)$$

$$u_i - u_j + nx_{ij} \leq n - 1 \quad \forall i \neq j, \quad i, j \in C \quad (3.4)$$

$$u_i \in \mathbb{R} \quad \forall i \in C \quad (3.5)$$

$$x_{ij} \in \{0, 1\} \quad \forall i, j \in V, i \neq j \quad (3.6)$$

The objective function (3.1) seeks to minimize the total cost of the route by summing costs of all the used edges. Constraints (3.2) and (3.3) ensure that all nodes have exactly one entering and one leaving edge. in (3.4) and (3.5) u_i is set to be the ordered number of node i , so the value of u_i of the depot will be zero, the first visited node will be one and so forth. These two constraints Introduced by Miller, Tucker and Zemlin (1960) (I still need to check wether I can citep this) prevent the creation of any subtours. Finally (3.6) specifies the decision variables.

3.2 Extension to the Flying Sidekick Traveling Salesman Problem

In the Flying Sidekick Traveling Salesman Problem, a set of customers must be served by a hybrid delivery system combining a truck and a drone. Both the truck and drone start at the depot. In this system the truck serves two roles, it can deliver packages in its traditional manner and secondly it acts as a loading and charging point for the drone while it is inactive. The drone will have the option to depart and rendezvous from the truck to serve one extra customer if the endurance of the drone allows for it. For simplicity, the department and rendezvous will only be possible at the depot and customer locations. Therefore, synchronization at these point of the truck and drone is required with the allowance of a waiting time for either vehicles. Additionally it is not allowed to revisit any nodes, this also includes for departures and returns of the drone. After landing on the truck, the battery of the drone is replaced and charged such that the endurance of the drone will remain the same for each customer visited by a drone. To account for service times of the drone a launch and recovery service time is also implemented. This introduces additional decisions:

- assignment of customers to truck or drone,
- launch and rendezvous locations,
- synchronization of truck and drone in time.

3.2.1 Model notation

Currently the MILP with the shortest computational time is from Boccia, M. et al. (Boccia, 2023) which is as follows;

Sets

- C : set of customers.
- $\{s, t\}$: Depot starding and ending node
- $V = C \cup \{s, t\}$: set of nodes.
- $A = \{(i, j) : i \in C \cup \{s\}, j \in C \cup \{t\}\}$: set of arcs.

Parameters

- t_{ij} : truck travel time on arc (i, j) .
- d_{ij} : drone travel time on arc (i, j) .
- Dtl : drone endurance.
- SL : launch service time.
- SR : recovery service time.

Decision variables

- $y_{ij} \in \{0, 1\}$: 1 if the truck travels arc (i, j) .
- $x_{ij} \in \{0, 1\}$: 1 if the drone flies along arc (i, j) .
- $\gamma_{ij}^h \in \{0, 1\}$: 1 if the truck travels arc (i, j) during the sortie serving customer h .
- $\theta_h \in \{0, 1\}$: 1 if customer h is served by the drone.
- $\omega_i^h \in \{0, 1\}$: 1 if node i is the launch node of the sortie serving h .
- $\delta_j^h \in \{0, 1\}$: 1 if node j is the rendezvous node of the sortie serving h .
- $\sigma_h \geq 0$: waiting time of the truck for sortie h .

Objective function

$$\min \sum_{(i,j) \in A} t_{ij} y_{ij} + \sum_{h \in C} (SL + SR) \theta^h - \sum_{h \in C} SL \omega_s^h + \sum_{h \in C \cup \{t\}} \sigma^h \quad (3.7)$$

Truck routing constraints

$$\sum_{j:(s,j) \in A} y_{sj} = \sum_{i:(i,t) \in A} y_{it} = 1 \quad (3.8)$$

$$\sum_{j:(i,j) \in A} y_{ij} = \sum_{j:(j,i) \in A} y_{ji} \leq 1 \quad \forall i \in C \quad (3.9)$$

$$\sum_{\substack{(i,j) \in A \\ i,j \in S}} y_{ij} \leq \sum_{h \in S \setminus \{q\}} (1 - \theta_h) \quad \forall 2 \subseteq S \subseteq V, q \in S \setminus \{s, t\} \quad (3.10)$$

Truck-drone linking constraints

$$\sum_{j:(s,j) \in A} \gamma_{sj}^h = \omega_s^h \quad \forall h \in C \quad (3.11)$$

$$\sum_{j:(i,t) \in A} \gamma_{it}^h = \delta_t^h \quad \forall h \in C \quad (3.12)$$

$$\sum_{j:(i,j) \in A} \gamma_{ij}^h - \sum_{j:(i,j) \in A} \gamma_{ji}^h = \omega_i^h - \delta_i^h \quad \forall i, h \in C \quad (3.13)$$

Assignment constraints

$$y_{sj} + x_{sj} \leq 1 \quad \forall (s, j) \in A \quad (3.14)$$

$$y_{it} + x_{it} \leq 1 \quad \forall (i, t) \in A \quad (3.15)$$

$$y_{ij} + x_{ij} + x_{ji} \leq 1 \quad \forall i, j \in C \quad (3.16)$$

$$\sum_{j:(h,j) \in A} y_{hj} + \theta^h = 1 \quad \forall h \in C \quad (3.17)$$

Consistency constraints

$$\sum_{h \in C} \gamma_{ij}^h \leq y_{ij} \quad \forall (i, j) \in A \quad (3.18)$$

$$\sum_{i \in V \setminus \{t, h\}} \omega_i^h = \sum_{j \in V \setminus \{s, h\}} \delta_j^h = \theta^h \quad \forall h \in C \quad (3.19)$$

$$x_{ij} \leq \theta^i + \theta^j \quad \forall (i, j) \in A \quad (3.20)$$

$$x_{ij} \leq \omega_i^j + \delta_j^i \quad \forall (i, j) \in A \quad (3.21)$$

$$\sum_{j:(i,j) \in A} x_{ij} = \sum_{h \in C \setminus \{i\}} \omega_i^h + \theta^i \leq 1 \quad \forall i \in C \quad (3.22)$$

$$\sum_{i:(i,j) \in A} x_{ij} = \sum_{h \in C \setminus \{j\}} \delta_j^h + \theta^j \leq 1 \quad \forall j \in C \quad (3.23)$$

Drone endurance constraints

$$\sum_{(i,j) \in A} t_{ij} \gamma_{ij}^h \leq (D^{tl} - SR) \theta_h \quad \forall h \in C \quad (3.24)$$

$$\sum_{i \in V \setminus \{t\}} d_{ih} \omega_i^h + \sum_{j \in V \setminus \{s\}} d_{hj} \delta_j^h \leq (D^{tl} - SR) \theta_h \quad \forall h \in C \quad (3.25)$$

Waiting time constraints

$$\sum_{i \in V \setminus \{t\}} d_{ih} \omega_i^h + \sum_{j \in V \setminus \{s\}} d_{hj} \delta_j^h - \sum_{(i,j) \in A} t_{ij} \gamma_{ij}^h \leq \sigma_h \quad \forall h \in C \quad (3.26)$$

Insert explanation of the model here:

Chapter 4

Heuristics

The FSTSP is an extension to the well known TSP, which is NP-hard, naturally this implies the FSTSP is NP-hard as well. Because of this even small customer sets consume a long time to compute. Therefore it is important to evaluate heuristics

4.1 Route and Reassign Heuristic

The heuristic proposed by Murray and Chu (Murray and Chu, 2015) is a route-and-reassign procedure. Some small adjustments in notation have been made for the sake of consistency.

It begins with a truck-only TSP solution and iteratively improves it by reassigning customers. At each step, the algorithm considers removing a customer from the truck route and either (i) assigning it to the UAV or (ii) reinserts it elsewhere in the truck route if it yields a positive net savings. The move that yields the greatest reduction in total delivery time is then applied and the process is repeated until no further improvements are possible.

4.1.1 Complete algorithmic framework

Algorithm 1 Main FSTSP heuristic

```

1: Initialize:
2:  $C' \leftarrow C$  ▷ Copy of the set of customers
3:  $[\text{truckRoute}, T] \leftarrow \text{solveTSP}(C)$ 
4:  $\text{truckSubRoutes} \leftarrow \{\text{truckRoute}\}$ 
5:  $\text{maxSavings} \leftarrow 0$ 
6: repeat
7:   for all  $h \in C'$  do
8:      $\text{CALCSAVINGS}(j, T)$ 
9:     for all subroute in  $\text{truckSubRoutes}$  do
10:      if there is a UAV associated with this subroute then
11:         $\text{CALCCOSTTRUCK}(h, T, \text{subroute})$ 
12:      else
13:         $\text{CALCCOSTUAV}(h, T, \text{subroute})$ 
14:      end if
15:    end for
16:  end for
17:  if  $\text{maxSavings} > 0$  then
18:     $\text{PERFORMUPDATE}$ 
19:     $\text{maxSavings} \leftarrow 0$ 
20:  else
21:    Stop
22:  end if
23: until Stop

```

Algorithm 2 $\text{CALCSAVINGS}(h, T)$

Require: h (a customer currently assigned to the truck), T (the vector of the truck's arrival time to each node)

```

1: Find  $i$ , the node immediately preceding  $j$  in the truck's route
2: Find  $j$ , the immediate successor node to  $h$  in the truck's route
3:  $\text{savings} \leftarrow s_{i,h} + s_{h,j} - s_{i,j}$ 
4: if  $h$  is currently in a truck subroute paired with the UAV then
5:   Find  $a$ , the first node in the truck's subroute (where the UAV launches)
6:   Find  $b$ , the last node in the truck's subroute (where the UAV returns)
7:   Find  $h'$ , the customer visited by the UAV associated with this subroute
8:   Calculate  $T'[b]$ , the truck's arrival time to  $b$  if  $j$  is removed from the truck route
9:    $\text{savings} \leftarrow \min \{\text{savings}, T'[b] - (T[a] + d_{a,h'} + d_{h',b} + SR)\}$ 
10: end if

```

Algorithm 3 CALCOSTTRUCK($h, T, \text{subroute}$)

Require: $h, T, \text{subroute}$

```
1: Find  $a$ , the first node in the truck's subroute
2: Find  $b$ , the last node in the truck's subroute
3: for all adjacent  $i$  and  $j$  in subroute do
4:    $\text{cost} \leftarrow t_{i,h} + t_{h,j} - t_{i,j}$ 
5:   if  $\text{cost} < \text{savings}$  then
6:     if  $t[b] - t[a] + \text{cost} \leq Dtl$  then
7:       if  $\text{savings} - \text{cost} > \text{maxSavings}$  then
8:          $\text{servedByUAV} \leftarrow \text{False}$ 
9:          $h^* \leftarrow h, i^* \leftarrow i, j^* \leftarrow j$ 
10:         $\text{maxSavings} \leftarrow \text{savings} - \text{cost}$ 
11:      end if
12:    end if
13:  end if
14: end for
```

Algorithm 4 CALCOSTUAV($h, T, \text{subroute}$)

Require: $h, T, \text{subroute}$

```
1: for all  $i$  and  $j$  in subroute, such that  $i$  precedes  $j$  do
2:   if  $d_{i,h} + d_{h,j} + SL + SR \leq Dtl$  then
3:     Find  $T'[j]$ , the truck's arrival time to node  $j$  if  $h$  were removed from
     the truck's route
4:      $\text{cost} \leftarrow \max \{0, \max (T'[k] - T[i] + SL + SR, d_{i,h} + d_{h,j} + SL + SR) - (T'[j] - T[i])\}$ 
5:     if  $\text{savings} - \text{cost} > \text{maxSavings}$  then
6:        $\text{servedByUAV} \leftarrow \text{True}$ 
7:        $i^* \leftarrow i, h^* \leftarrow h, j^* \leftarrow j$ 
8:        $\text{maxSavings} \leftarrow \text{savings} - \text{cost}$ 
9:     end if
10:  end if
11: end for
```

Algorithm 5 PERFORMUPDATE($\text{servedByUAV}, i^*, h^*, j^*$)

Require: $\text{servedByUAV}, i^*, h^*, j^*$

```
1: if  $\text{servedByUAV} = \text{True}$  then
2:   Assign the UAV to sortie  $i^* \rightarrow h^* \rightarrow j^*$ 
3:   Remove  $h^*$  from  $\text{truckRoute}$  and  $\text{truckSubRoutes}$ 
4:   Append a new truck subroute that starts at  $i^*$  and ends at  $j^*$ 
5:   Remove  $i^*, h^*$ , and  $j^*$  from  $C'$ 
6:   Update  $T$  (the vector of truck arrival times)
7: else
8:   Remove  $h^*$  from its current truck subroute
9:   Insert  $h^*$  between  $i^*$  and  $j^*$  in the new truck subroute
10:  Update  $\text{truckRoute}$  to reflect the new sequence of nodes visited
11:  Update  $T$ 
12: end if
```

4.1.2 Explanation of the heuristic

Initially, all customers are assigned to the truck. This route is stored as a set of truck subroutes, which later become important when UAV sorties are introduced.

For each UAV-eligible customer $h \in C$, the procedure computes the *savings* obtained by removing that customer from its current position in the truck route; see Algorithm 2. If h lies in a subroute that is already coordinated with a UAV sortie, then the savings may be limited by the synchronization requirement between truck and UAV. In that case, the savings are adjusted to reflect the earliest feasible return of the UAV.

After the savings are computed, the heuristic evaluates two alternative ways of reallocating customer h .

First, if a given truck subroute is already associated with a UAV sortie, then customer h may only be considered for reinsertion into the truck route. This is handled by Algorithm 3. The customer is tentatively inserted between adjacent nodes of the selected subroute, and the additional truck cost is calculated. Such an insertion is only feasible if the UAV assigned to that subroute can still complete its sortie within the endurance limit Dtl . If the resulting net improvement, namely savings minus insertion cost, is the best encountered so far, then this move is stored.

Second, if the considered subroute is not currently associated with a UAV, then customer h may instead be served by the UAV. This is handled by Algorithm 4. For every ordered pair of nodes (i, j) in the subroute, with i preceding j , the heuristic checks whether the UAV sortie $i \rightarrow h \rightarrow j$ is feasible with respect to endurance. If so, it calculates the cost of launching and recovering the UAV, together with any waiting time caused by synchronization with the truck. Again, if the net improvement exceeds the best previously found value, the corresponding UAV assignment is stored.

Once all eligible customers and all truck subroutes have been examined, the best improving move is implemented through Algorithm 5. If the best move

is a UAV assignment, then customer h^* is removed from the truck route and a new truck subroute is created between the launch node i^* and the rendezvous node j^* . If the best move is instead a truck reinsertion, then h^* is simply moved to a new position in the truck route. In both cases, the truck arrival times are updated.

The procedure repeats until no reassignment yields positive savings. At that point, the algorithm terminates. Hence, the final solution is locally improved relative to the initial truck-only TSP route, while explicitly accounting for UAV endurance and truck-UAV synchronization.

4.1.3 Example

To illustrate the heuristic, a small example instance is considered. The following parameters are used:

Example parameters

- t_{ij} : truck travel time between nodes i and j , as given in Figure 4.1b.
- d_{ij} : drone travel time between nodes i and j , defined as

$$d_{ij} = \frac{1}{2}t_{ij}, \quad \forall i, j,$$

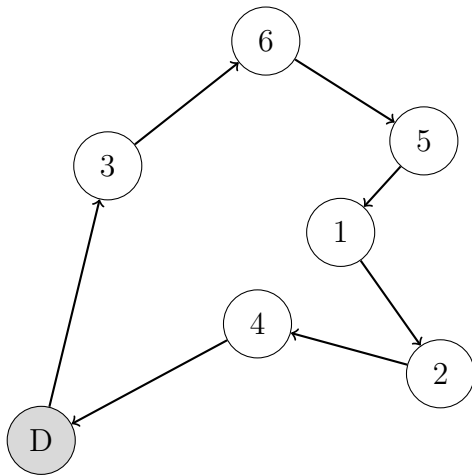
implying that the drone travels twice as fast as the truck.

- SL and SR : launch and recovery service times, respectively, both set to

$$SL = SR = 0.2.$$

- e : drone endurance, set to

$$e = 5.$$



(a) Node layout

	D	1	2	3	4	5	6
D	—	4.4	4.9	3.4	3.0	5.8	5.5
1	4.4	—	2.0	3.0	1.5	1.5	2.5
2	4.9	2.0	—	4.9	2.3	2.8	4.5
3	3.4	3.0	4.9	—	2.6	3.9	2.4
4	3.0	1.5	2.3	2.6	—	3.0	3.4
5	5.8	1.5	2.8	3.9	3.0	—	2.3
6	5.5	2.5	4.5	2.4	3.4	2.3	—

(b) Distance matrix

Figure 4.1: Example Heuristic.

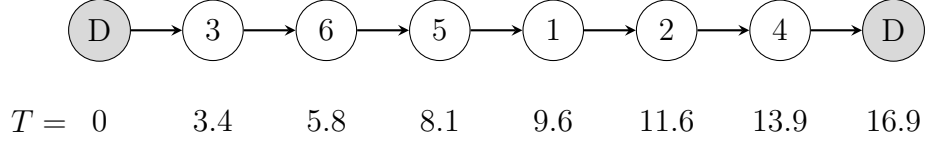


Figure 4.2: Optimal TSP route represented as an ordered sequence of nodes with corresponding arrival times.

We start with the first iteration of the heuristic. Consider customer $j = 1$, which is located between nodes 5 and 2 in the current truck route. The potential saving from removing node 1 from the truck route is computed using Algorithm 2 and is given by

$$t_{5,1} + t_{1,2} - t_{5,2} = 1.5 + 2.0 - 2.8 = 0.7.$$

Since node 1 is not yet part of a subroute assigned to a UAV, we proceed to Algorithm 4, where all feasible launch and rendezvous nodes (i, k) are evaluated for serving node 1 by the drone.

For each pair (i, k) , feasibility is first checked using the endurance constraint:

$$d_{i,1} + d_{1,k} + S_L + S_R \leq e.$$

The lowest-cost option is obtained by launching the drone at node 5 and retrieving it at node 2. After removing node 1 from the truck route, the updated truck arrival time at node 2 becomes

$$T'[2] = T[5] + t_{5,2} = 8.1 + 2.8 = 10.9.$$

The cost of assigning customer 1 to the UAV between nodes 5 and 2 is then computed using Algorithm 4 as

$$\max \{0, \max (T'[2] - T[5] + S_L + S_R, d_{5,1} + d_{1,2} + S_L + S_R) - (T'[2] - T[5])\}.$$

Substituting the numerical values gives

$$\max \{0, \max ((10.9 - 8.1) + 0.2 + 0.2, 0.75 + 1.00 + 0.2 + 0.2) - (10.9 - 8.1)\}.$$

This simplifies to

$$\max \{0, \max(3.2, 2.15) - 2.8\} = \max\{0, 0.4\} = 0.4.$$

Thus, serving customer 1 by UAV between nodes 5 and 2 has a cost of 0.4. Since the saving from removing customer 1 is 0.7, the resulting net improvement is

$$0.7 - 0.4 = 0.3.$$

The computed value of the objective, denoted as *MaxSaving*, is updated whenever a higher value is found.

Following this evaluation, the largest improvement is achieved by assigning the UAV to the sortie $5 \rightarrow 2 \rightarrow 4$.

The solution is then updated accordingly. Subroutes are introduced to represent the segments during which the UAV is active. In addition, the launch node, the rendezvous node, and the drone-served customer are removed from the set of eligible customers for subsequent iterations, yielding

$$C' = \{3, 6, 1\}.$$

The resulting configuration is illustrated in Figure 4.3.

Finally, it should be noted that service times are incorporated in Figure 4.3 for illustrative purposes. However, this representation does not fully correspond to the definition of T , which only denotes the arrival time at each node.

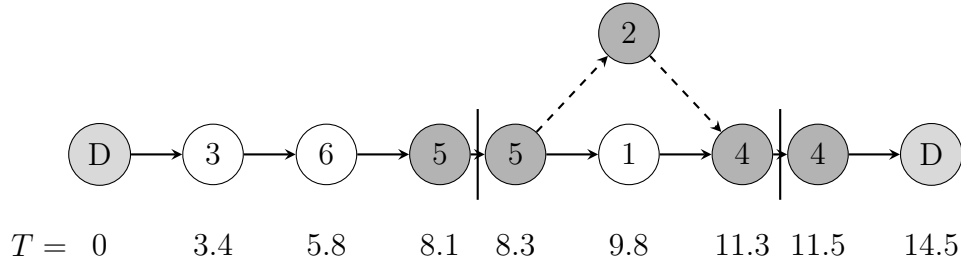


Figure 4.3: Updated route with UAV assignment $5 \rightarrow 2 \rightarrow 4$. Nodes 5, 2, and 4 are shaded to indicate infeasibility for future iterations. Vertical separators indicate subroute boundaries.

Now consider the next iteration. At this stage, a UAV subroute has already been established, while customer 1 remains in the set of eligible nodes and can still be considered for reassignment. To evaluate this possibility, we again compute the saving associated with removing node 1 from its current position. However, unlike in the first iteration, the savings function must now account for the presence of an existing UAV subroute. In particular, it incorporates the potential change in synchronization at the rendezvous node, as described in line 11 of Algorithm 2.

The adjusted saving is obtained by taking the minimum of two quantities: the standard truck saving and the saving that results from a reduced waiting time of the UAV. In this example, the standard saving is

$$t_{5,1} + t_{1,4} - t_{5,4} = 1.5 + 1.5 - 3 = 0,$$

while the synchronization-based saving is given by

$$11.3 - (8.1 + 1.4 + 1.55 + 0.2) = 0.45.$$

Although the second term indicates that the UAV would have to wait less for the truck, the overall saving is determined by the minimum of the two values, which yields

$$\text{savings} = \min\{0, 0.45\} = 0.$$

The reason for this is that node 1 lies directly between nodes 5 and 4, so removing it does not reduce the truck travel time. Consequently, even though

some synchronization improvement is possible, it does not translate into a net reduction in the overall route duration. Because the resulting saving is zero, we can conclude that relocating node 1 will not lead to any improvement.

Now considering the remaining eligible customers, a final improvement can be achieved by removing node 6 from the truck route and assigning it to the UAV via the sortie $3 \rightarrow 6 \rightarrow 5$, as illustrated in Figure 4.4. This move is obtained using the same procedure as in the first iteration.

Since no further improvements are possible, this step concludes the heuristic.

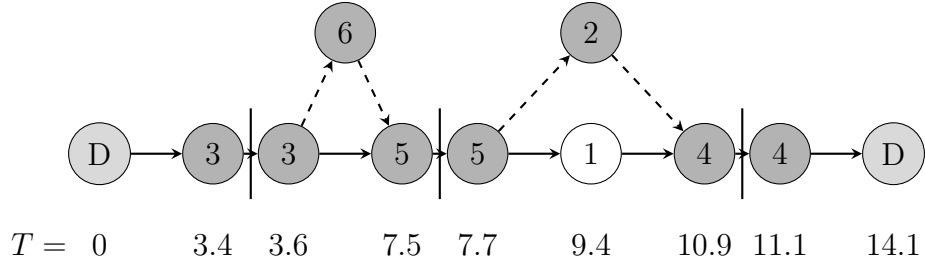


Figure 4.4: Updated route after adding the UAV sortie $3 \rightarrow 6 \rightarrow 5$.

Heuristic evaluation

This heuristic works straight forward, and locks certain nodes with each iteration, however this could lead to locking in nodes that could still be put in more optimal solution. Lets take another look at our examples finale route and compare it with the actual optimal route obtained by our MILP. We can see some complete differences between the two, this leads us to investigate in an additional method described in the next section

4.2 Simulated Annealing heuristic

The second heuristic considered in this thesis is based on Simulated Annealing. In contrast to the heuristic of Murray and Chu, this method does not evaluate all possible reassignment moves in order to implement the best positive improvement. Instead, it repeatedly generates a random modification of the current solution and then decides whether this new solution should be accepted.

The idea behind Simulated Annealing is inspired by the cooling process of physical systems. A system starts at a high temperature, where many different configurations are possible, and is then gradually cooled down. If this cooling process is slow enough, the system tends to move towards a low-energy state. In optimization, this idea is translated to the search for a solution with a low objective value. For a routing problem, each feasible route can be viewed as a configuration, while the corresponding completion time represents its energy.

In an Operations Research context, Simulated Annealing explores the solution space by repeatedly generating neighbouring solutions. These neigh-

bouring solutions are obtained by applying small modifications, called moves, to the current solution. If the new solution improves the objective value, it is accepted. If the new solution is worse, it may still be accepted with a certain probability. This probability depends on the deterioration in objective value and on the current temperature. At high temperatures, worse solutions are more likely to be accepted. As the temperature decreases, the algorithm becomes more selective and increasingly behaves like a local improvement method.

This acceptance mechanism allows the algorithm to escape local optima. A local optimum is a solution that cannot be improved by a small local change, even though better solutions may exist elsewhere in the solution space. By sometimes accepting a worse solution, Simulated Annealing can move away from such a point and continue the search in a different region.

The Simulated Annealing heuristic implemented in this thesis is based on the heuristic of Ponza, 2016, but with a simplified solution representation and solution evaluation. Instead of computing separate savings expressions for every possible move, each tentative solution is evaluated completely. This makes the implementation easier to verify and reduces the number of case distinctions required in the algorithm.

4.2.1 Solution representation

A solution is represented by two components. The first component is the truck route, denoted by

$$R = (r_0, r_1, \dots, r_m),$$

where $r_0 = s$ and $r_m = t$ denote the starting and ending depot, respectively. The second component is the set of UAV sorties, denoted by

$$\mathcal{S} = \{(i, h, j)\}.$$

In each sortie (i, h, j) , node i is the launch node, customer h is served by the UAV, and node j is the rendezvous node. A complete solution is therefore denoted by

$$S = (R, \mathcal{S}).$$

Each customer must be served exactly once. Therefore, a customer either appears in the truck route or appears as the UAV-served customer in one sortie. This can be written as

$$C = (R \setminus \{s, t\}) \cup \{h : (i, h, j) \in \mathcal{S}\},$$

where the two sets in the union must be disjoint. Since all customers are assumed to be serviceable by the UAV, no separate UAV-eligible customer set is used.

4.2.2 Main algorithm

The Simulated Annealing heuristic starts from an initial feasible solution S^0 . In the standalone version of the heuristic, this is the truck-only TSP solution:

$$S^0 = (R^{TSP}, \emptyset).$$

This gives the Simulated Annealing heuristic the same starting point as the heuristic of Murray and Chu.

Let S^{cur} denote the current solution, S^{new} the tentative solution, and S^{best} the best solution found so far. The objective value of a solution S is denoted by $Z(S)$ and represents the total completion time.

The start and final temperatures are initialized as fractions of the initial objective value:

$$T_s = \tau_s Z(S^0), \quad T_f = \tau_f Z(S^0).$$

Given a fixed number of iterations N , the cooling rate is calculated as

$$\alpha = \exp\left(\frac{\log(T_f/T_s)}{N}\right).$$

After each iteration, the temperature is updated by

$$T \leftarrow \alpha T.$$

The complete procedure is given in Algorithm 6.

Algorithm 6 Main Simulated Annealing heuristic

```
1: Initialize:
2:  $S^{cur} \leftarrow S^0$ 
3:  $S^{best} \leftarrow S^0$ 
4:  $Z^{cur} \leftarrow \text{EvaluateSolution}(S^{cur})$ 
5:  $Z^{best} \leftarrow Z^{cur}$ 
6:  $T_s \leftarrow \tau_s Z^{cur}$ 
7:  $T_f \leftarrow \tau_f Z^{cur}$ 
8:  $\alpha \leftarrow \exp\left(\frac{\log(T_f/T_s)}{N}\right)$ 
9:  $T \leftarrow T_s$ 
10: for  $q = 1, \dots, N$  do
11:    $S^{new} \leftarrow \text{copy of } S^{cur}$ 
12:    $\text{ApplyRandomMove}(S^{new})$ 
13:    $feasible \leftarrow \text{FeasibilityCheck}(S^{new})$ 
14:   if  $feasible = \text{True}$  then
15:      $Z^{new} \leftarrow \text{EvaluateSolution}(S^{new})$ 
16:      $\Delta \leftarrow Z^{new} - Z^{cur}$ 
17:     if  $\Delta \leq 0$  then
18:        $S^{cur} \leftarrow S^{new}$ 
19:        $Z^{cur} \leftarrow Z^{new}$ 
20:     else
21:       Generate  $u \sim U(0, 1)$ 
22:       if  $u \leq \exp(-\Delta/T)$  then
23:          $S^{cur} \leftarrow S^{new}$ 
24:          $Z^{cur} \leftarrow Z^{new}$ 
25:       end if
26:     end if
27:     if  $Z^{cur} < Z^{best}$  then
28:        $S^{best} \leftarrow S^{cur}$ 
29:        $Z^{best} \leftarrow Z^{cur}$ 
30:     end if
31:   end if
32:    $T \leftarrow \alpha T$ 
33: end for
34: return  $S^{best}$ 
```

The value Δ is computed in every iteration, since it represents the difference between the current solution and the newly generated tentative solution. If $\Delta \leq 0$, the tentative solution improves the current solution and is accepted. If $\Delta > 0$, the solution is worse and is accepted with probability

$$\exp\left(-\frac{\Delta}{T}\right).$$

This probability becomes smaller as the temperature decreases.

4.2.3 Move operators

The Simulated Annealing heuristic uses four move operators. At each iteration, one of these moves is selected randomly. The move operators are shown in Algorithms 7–11.

Algorithm 7 ApplyRandomMove(S)

```
1: Randomly select a move type  $m$ 
2: if  $m = \text{truck-to-truck relocation}$  then
3:   TruckToTruck( $S$ )
4: else if  $m = \text{truck-to-UAV conversion}$  then
5:   TruckToUAV( $S$ )
6: else if  $m = \text{UAV-to-truck conversion}$  then
7:   UAVToTruck( $S$ )
8: else if  $m = \text{change sortie}$  then
9:   ChangeSortie( $S$ )
10: end if
```

Algorithm 8 TruckToTruck(S)

```
1: Select a random truck customer  $h \in R \setminus \{s, t\}$ 
2: if  $h$  is not a launch or rendezvous node of any sortie then
3:   Remove  $h$  from its current position in  $R$ 
4:   Select a random insertion position in  $R$ 
5:   Insert  $h$  at the selected position
6: end if
```

Algorithm 9 TruckToUAV(S)

```
1: Select a random truck customer  $h \in R \setminus \{s, t\}$ 
2: if  $h$  is not a launch or rendezvous node of any sortie then
3:   Remove  $h$  from  $R$ 
4:   Select two random nodes  $i, j \in R$  such that  $i$  occurs before  $j$ 
5:   Add sortie  $(i, h, j)$  to  $\mathcal{S}$ 
6: end if
```

Algorithm 10 UAVToTruck(S)

```
1: if  $\mathcal{S} \neq \emptyset$  then
2:   Select a random sortie  $(i, h, j) \in \mathcal{S}$ 
3:   Remove  $(i, h, j)$  from  $\mathcal{S}$ 
4:   Select a random insertion position in  $R$ 
5:   Insert  $h$  at the selected position
6: end if
```

Algorithm 11 ChangeSortie(S)

```
1: if  $S \neq \emptyset$  then  
2:   Select a random sortie  $(i, h, j) \in S$   
3:   Remove  $(i, h, j)$  from  $S$   
4:   Select two random nodes  $i', j' \in R$  such that  $i'$  occurs before  $j'$   
5:   Add sortie  $(i', h, j')$  to  $S$   
6: end if
```

The truck-to-truck move changes only the order of the truck route. A truck customer is removed from its current position and inserted elsewhere. Launch and rendezvous nodes are not selected in this move, because changing such nodes would also affect an existing UAV sortie.

The truck-to-UAV move removes a truck customer from the truck route and assigns it to the UAV. A launch node and rendezvous node are then selected from the remaining truck route. This move allows the heuristic to create new UAV deliveries.

The UAV-to-truck move does the opposite. It removes an existing sortie and inserts the UAV-served customer back into the truck route. This is necessary because the algorithm must be able to undo earlier UAV assignments.

The change-sortie move keeps the same UAV-served customer, but changes the launch and rendezvous nodes. This allows the algorithm to improve an existing sortie without first moving the customer back to the truck.

4.2.4 Feasibility check and evaluation

After a move has been applied, the tentative solution is checked for feasibility. This is done after the move instead of during the move itself, so the move operators can remain simple.

Algorithm 12 FeasibilityCheck(S)

```
1:  $C^T \leftarrow R \setminus \{s, t\}$ 
2:  $C^U \leftarrow \{h : (i, h, j) \in \mathcal{S}\}$ 
3: if  $C^T \cap C^U \neq \emptyset$  then
4:   return False
5: end if
6: if  $C^T \cup C^U \neq C$  then
7:   return False
8: end if
9: for all sorties  $(i, h, j) \in \mathcal{S}$  do
10:   if  $i \notin R$  or  $j \notin R$  then
11:     return False
12:   end if
13:   if  $i$  does not occur before  $j$  in  $R$  then
14:     return False
15:   end if
16:   Compute
17:   if  $L_{i,h,j}(S) > Dtl$  then
18:     return False
19:   end if
20: end for
21: if two sorties overlap in the truck route then
22:   return False
23: end if
24: return True
```

$$L_{i,h,j}(S) = SL + \max\{T_{i,j}^R, d_{i,h} + d_{h,j}\} + SR$$

Algorithm 12 first checks whether every customer is served exactly once. It then checks whether each UAV sortie has a valid launch and rendezvous node, whether the launch occurs before the rendezvous, and whether the endurance limit Dtl is respected. Lastly, it checks that no two sorties overlap, since only one UAV is available.

Finally algorithm 13 evaluates the complete truck route and returns. For two truck nodes r_ℓ and r_k , with $\ell < k$, let

$$T_{r_\ell, r_k}^R = \sum_{a=\ell}^{k-1} t_{r_a, r_{a+1}}$$

denote the truck travel time between these nodes along the current truck route. Furthermore, for a sortie (r_ℓ, h, r_k) , let

$$D_{r_\ell, h, r_k} = d_{r_\ell, h} + d_{h, r_k}$$

denote the corresponding UAV flight time.

Algorithm 13 EvaluateSolution(S)

```
1:  $Z \leftarrow 0$ 
2:  $\ell \leftarrow 0$ 
3: while  $\ell < |R| - 1$  do
4:   Let  $r_\ell$  be the current truck node
5:   if there exists a sortie  $(r_\ell, h, r_k) \in \mathcal{S}$  starting at  $r_\ell$  then
6:      $Z \leftarrow Z + SL + \max\{T_{r_\ell, r_k}^R, D_{r_\ell, h, r_k}\} + SR$ 
7:      $\ell \leftarrow k$ 
8:   else
9:      $Z \leftarrow Z + t_{r_\ell, r_{\ell+1}}$ 
10:     $\ell \leftarrow \ell + 1$ 
11:   end if
12: end while
13: return  $Z$ 
```

4.3 Hybrid Simulated Annealing heuristic

The Simulated Annealing heuristic can also be used as a post-optimization method. Instead of starting from the truck-only TSP solution, it can start from the solution obtained by the route-and-reassign heuristic of (Murray and Chu, 2015). This gives a hybrid heuristic.

Algorithm 14 Hybrid Simulated Annealing heuristic

```
1: Initialize:
2: Compute initial truck-only TSP route  $R^{TSP}$ 
3: Apply the heuristic of Murray and Chu to obtain  $S^{MC}$ 
4:  $S^0 \leftarrow S^{MC}$ 
5: Apply Algorithm 6 with initial solution  $S^0$ 
6: return  $S^{best}$ 
```

The only difference between the standalone Simulated Annealing heuristic and the hybrid version is the initial solution. In the standalone version,

$$S^0 = (R^{TSP}, \emptyset),$$

whereas in the hybrid version,

$$S^0 = S^{MC}.$$

Here, S^{MC} denotes the solution obtained from the heuristic of Murray and Chu.

The purpose of this approach is to test whether the Simulated Annealing procedure can further improve the solution produced by the first heuristic. The hybrid heuristic should therefore not be interpreted as a fully independent heuristic. Rather, it is an improvement procedure applied after a constructive FSTSP solution has already been obtained. If the hybrid heuristic

performs well, this indicates that Simulated Annealing can be useful as a post-optimization method. However, it must be noted that this hybrid solution will always out-perform the route-and-reassign heuristic due to it's strictly optimizing behavior.

Chapter 5

Experimental Setup

Before presenting the results, the experimental setup is described. This includes the generated customer instances, the operational parameters such as truck speed, UAV speed, endurance and service times, and the algorithmic parameters of the Simulated Annealing heuristic. These choices are made to reflect realistic delivery conditions while remaining comparable to previous truck–UAV routing studies.

5.1 Base model

For the computational experiments, customer instances are generated randomly within a square delivery area with a side length of 30 km. This distance range is chosen in relation to the assumed drone endurance of 15 km. Two spatial settings are considered. In the first setting, the depot is located in the centre of the delivery area. In the second setting, the depot is located in one of the corners of the delivery area. This makes it possible to evaluate how the relative position of the depot influences the performance of the truck–UAV system.

Truck travel times are computed from Euclidean distances, but adjusted to reflect the fact that trucks are restricted to the road network. A delivery truck is assumed to travel at an average speed of 30 km/h (Lyu, 2021). In addition, actual road distances are approximated as 1.3 times the Euclidean distance (Wang, 2024). Therefore, if the Euclidean distance between two nodes is 1 km, the corresponding truck travel distance is approximated by

$$1.3 \cdot 1 = 1.3 \text{ km.}$$

At a speed of 30 km/h, this results in a travel time of

$$\frac{1.3}{30} \cdot 3600 = 156 \text{ seconds.}$$

Hence, the truck travel time is set to 156 seconds per kilometer of Euclidean distance.

For the UAV, the Workhorse HorseFly is used as a reference vehicle. Its cruise speed is approximately 55.6 km/h, which corresponds to about 64.7

seconds per kilometer. To account for acceleration, deceleration, operational interference, and to keep the implementation simple, the UAV travel time is set to 78 seconds per kilometer. This implies that, in the computational experiments, the UAV travels twice as fast as the truck:

$$d_{ij} = \frac{1}{2}t_{ij}.$$

5.2 Simulated Annealing

Now for the simulated annealing, I actually still need to fuck around a little bit

Chapter 6

Results

Chapter 7

Conclusion

Bibliography

- Agatz, N., Bouman, P., and Schmidt, M. (2018). Optimization approaches for the traveling salesman problem with drone. *Transportation Science*, 52(4):965–981.
- Boccia, M. e. a. (2023). A new milp formulation for the flying sidekick traveling salesman problem. *European Journal of Operational Research*.
- De Kuyffer, J. e. a. (2024). Hybrid truck-drone delivery systems with coordinated routing. *Computers & Operations Research*.
- Es Yurek, E. E. and Ozmutlu, H. C. (2018). A decomposition-based iterative optimization algorithm for the traveling salesman problem with drone. *Transportation Research Part C: Emerging Technologies*, 91:249–262.
- Kirkpatrick, S. e. a. (1983). Optimization by simulated annealing. *Science*, 220:671–680.
- Li, Y. e. a. (2022). Truck-drone routing with operational constraints and coordination. *Transportation Research Part E: Logistics and Transportation Review*.
- Lyu, Z. e. a. (2021). Freight operations modelling for urban delivery and pickup with flexible routing: Cluster transport modelling incorporating discrete-event simulation and gis. *Infrastructures*, 6.
- Murray, C. C. and Chu, A. G. (2015). The flying sidekick traveling salesman problem: Optimization of drone-assisted parcel delivery. *Transportation Research Part C: Emerging Technologies*, 54:86–109.
- Najy, W. e. a. (2023). Multi-drone truck routing problem with synchronization constraints. *European Journal of Operational Research*.
- Ponza, A. (2016). Optimization of drone-assisted parcel delivery.
- Salama, M. and Srinivas, S. e. a. (2022). Collaborative truck multi-drone routing and scheduling problem. *Transportation Research Part C: Emerging Technologies*.
- Stern, J. (2013). Like amazon, ups also considering using unmanned flying vehicles. [://abcnews.go.com/Technology/amazon-ups-drone-delivery-options/story?id=21086160](https://abcnews.go.com/Technology/amazon-ups-drone-delivery-options/story?id=21086160).

- Wang, S. e. a. (2024). How much longer do you have to drive than the crow has to fly? *npj Complexity*, 1.
- Zhang, Y. e. a. (2022). Multi-objective optimization for drone-assisted delivery systems. *Transportation Research Part E: Logistics and Transportation Review*.

Appendix A

Extra Tables and Figures